

I. LES LIMITES D'UN TABLEAU COMME STRUCTURE DE DONNEES

Une **structure** de données permet de "**ranger**" des données en mémoire.

Ex 1 : Un tableau d'entiers permet de ranger de manière ordonnée plusieurs entiers.



D'un langage de programmation à l'autre, les structures de données qui existent peuvent être assez différentes. Mais il existe des structures de données dont le comportement est défini indépendamment du langage (on parle de type abstrait de données). On décrira, entre autres, cette année les listes chaînées, les piles et les files. Le choix de la structure de données doit être adapté à la nature de la situation à modéliser.



La structure de tableau permet de stocker des séquences d'éléments mais n'est pas adaptée à toutes les opérations que l'on peut souhaiter effectuer sur des séquences.



Les tableaux python (de classe '**list**') permettent par exemple d'insérer ou supprimer efficacement des éléments à la fin d'un tableau avec les opérations `append` et `pop` :

```
tableau = [10,20,30]
print(type(tableau))      # <class 'list'>
tableau.append(100)
print(tableau)             # [10, 20, 30, 100]
tableau.pop()
print(tableau)             # [10, 20, 30]
tableau.pop()
print(tableau)             # [10, 20]
```

Exo 1. [listes_0.py](#)

- Ecrire une fonction **concatener**(t1,t2) qui retourne la liste t1+t2 dans une troisième liste.
- Ecrire une fonction **insérer**(t,i,v) qui insère, **en place** (c'est-à-dire dans le tableau t et sans créer de tableau supplémentaire), la valeur v à l'indice i dans le tableau t.

Quel est le coût **C(n)** (complexité : on peut compter le nombre d'accès en écriture au tableau t) de l'opération **insérer**(t,0,v) si n est la taille du tableau t

La structure de liste chaînée apporte une réponse au coût élevé de ces deux opérations.

Rq1 L'opérateur **+** et la méthode **insert** de la classe '**list**' de Python permettent de faire ces deux opérations mais sans faire mieux en termes de complexité que celles implémentées à la main à l'exo précédent.

```
print(tableau)             # [10, 20]
tableau.insert(0,1000)
print(tableau)             # [1000,10, 20]
```



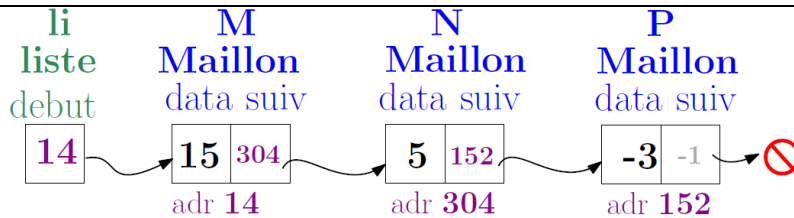
une liste en Python est... un mélange entre un tableau et une liste chaînée !

II. STRUCTURE DE LISTE CHAINEE

1. VISUALISATION (SCHEMATIQUE)

Une liste chaînée peut être vue comme un ensemble de "maillons" ou « cellules » dont on connaît l' "adresse" du premier maillon. Chaque maillon contient une donnée à stocker (par exemple, un entier) et un "lien" vers le maillon suivant.

Ex 2 : La liste li d'entiers, [15 ;5 ;-3] peut être vue en tant que liste chaînée, de la manière suivante :



- Rq2.** Physiquement, les maillons ne sont **pas stockés les uns à la suite des autres** en mémoire (au contraire d'un tableau qui occupera basiquement une zone de mémoire « sans trous »)
- Rq3.** Le "lien" vers le maillon suivant est l'adresse de l'emplacement mémoire où est stocké le maillon suivant ($\approx$ pointeur).
- Rq4.** Le lien du dernier maillon vers son suivant signale la fin de la liste : il pointe sur ... rien (**None** en python)
- Rq5.** Si la liste **li** est vide le début pointe sur ... rien (**None** en python donc **li == None**)

2. CLASSE MAILLON



On utilise une classe décrivant les maillons de la liste : chaque élément de la liste est donc une instance de la classe maillon.

class Maillon:

```
def __init__(self, v, s):
    self._valeur = v
    self._suivant = s
```

#retourne la valeur du maillon

```
def valeur(self):
    return self._valeur
```

#retourne la queue de la liste

```
def suite(self):
    return self._suivant
```

#methode de classe (partagée par toutes les instances)

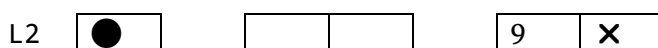
```
def est_vide(lst):
    return self is None
```

Exo 2. [classe maillon 0.py](#)

1. Construire, en utilisant la classe Maillon, la liste L1 : [9]



2. Construire la liste L2 : [6,9]



La valeur contenue dans la variable **L2** est l'adresse mémoire du premier maillon, qui lui-même dans son attribut **_suivant** contient l'adresse du deuxième maillon.

3. Construire la liste L3 : [3,6,9]



4. Proposer la construction sur une seule ligne de la liste L4 : [30,44,50] :

Définition réursive :

Une liste est donc soit la valeur **None** pour désigner une liste vide, soit un objet de la classe **Maillon** dont l'attribut **_valeur** contient la valeur de son premier élément (la *tête* de la liste) et l'attribut **_suivant** contient une liste (la *queue* de la liste)

Rq6. D'autres représentations de la classe **Maillon** sont possibles :

On pourrait représenter un maillon

- par un couple, et dans ce cas la construction de la liste **L4** devient :
L4= (30, (44, (50, None)))
- par un tableau de taille 2, et dans ce cas la construction de la liste **L4** devient :
L4= [30, [44, [50, None]]]

Mais l'utilisation des attributs valeur et suivant dans un objet instance de la classe **Maillon** rend le code plus lisible

Rq7. Comment faire évoluer la structure de la liste chaînée pour en faire une liste cyclique ?

III. OPERATIONS SUR LES LISTES



Quelques opérations sur les listes que l'on pourra programmer dans deux versions, l'une itérative, et l'autre réursive.



[classe maillon.py](#)

1. LONGUEUR D'UNE LISTE

a) Version itérative



Ne pas hésiter à simuler schématiquement l'exécution de l'algorithme avant de coder :

lst

●

15	●
----	---

5	●
---	---

12	×
----	---

maillon :

taille :



Positionner un curseur sur le premier maillon, pointé par **lst**

Puis, tant qu'il reste un maillon, parcourir la suite de la liste en comptant les maillons parcourus.

Exo 3.

- Coder en Python la fonction **taille_iter(lst)** qui retourne la longueur de la liste **lst**. Tester.
- Estimer la complexité de cette fonction en fonction de n, la taille de la liste.

b) Version récursive

Exo 4.

c. Compléter : $taille(lst) = \begin{cases} 0 & \text{si} \\ 1 + taille(\quad) & \text{sinon} \end{cases}$

d. Coder en Python la fonction **taille_rec(lst)** qui retourne la longueur de la liste **lst**. Tester.

2. N-IEME ELEMENT D'UNE LISTE



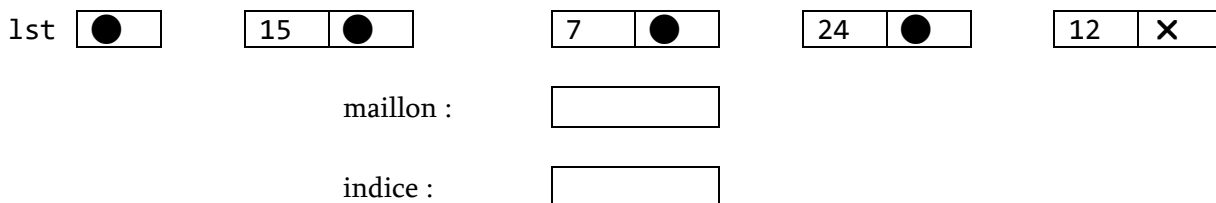
On souhaite coder une fonction **nieme_element(n, lst)** qui renvoie le **n**-ième élément d'une liste **lst**

a) Version itérative



Ne pas hésiter à simuler schématiquement l'exécution de l'algorithme avant de coder :

Ex 3. : Simulation de l'exécution de



Positionner un curseur sur le premier maillon, pointé par **lst**
 Puis, tant que l'on n'a pas parcouru **n** maillons, parcourir la suite de la liste.
 Retourner la valeur du dernier maillon parcouru.

Exo 5.

- Coder en Python la fonction **nieme_element_iter(n, lst)** qui retourne le **n**-ième élément de la liste **lst**. On lèvera une exception **IndexError** si la liste **lst** est vide ou si l'indice **n** n'est pas valide. Tester.
- Estimer la complexité de cette fonction.

b) Version récursive

Exo 6.

a. Compléter : $nieme_element(n, lst) = \begin{cases} \quad & \text{si } n = 0 \\ nieme_element(\quad) & \text{sinon} \end{cases}$

b. Coder en Python la fonction **nieme_element_rec(n, lst)** qui retourne le **n**-ième élément de la liste **lst**. On lèvera une exception **IndexError** si la liste **lst** est vide. Tester.

c. Vérifier le comportement de la fonction lorsque l'indice **n** n'est pas valide.

3. CONCATENER DEUX LISTES

Exo 7.

a. Implémenter la fonction suivante :

def concatener(lst1, lst2):

"""retourne une nouvelle liste lst3 qui concatène les listes lst1 et lst2 """



Les listes **lst1** et **lst2** ne doivent pas être modifiées : les éléments de **lst1** seront copiés, et ceux de **lst2** seront partagés.

Exo 8.

lst1

●

10	●
----	---

20	●
----	---

30	×
----	---

 lst2

●

40	●
----	---

50	●
----	---

60	×
----	---

lst3

- b. Une version récursive pourra se déduire de la formulation suivante à compléter :

$$\text{concatener}(\text{lst1}, \text{lst2}) = \begin{cases} \text{Maillon} \left(\text{lst1.valeur}, \right. & \text{si lst1 est vide} \\ & \left. \text{concatener}(\text{lst1.prochain}, \text{lst2}) \right) & \text{sinon} \end{cases}$$

Implémenter cette version.

- c. Estimer la complexité de la fonction implémentée.

4. OPERATIONS A RISQUES

Exo 9. On s'autorise maintenant à implémenter la fonction **concatener(lst1, lst2)** en modifiant l'une ou l'autre des listes **lst1** et **lst2** (ou les deux).

lst1

●

10	●
----	---

20	●
----	---

30	×
----	---

 lst2

●

40	●
----	---

50	●
----	---

60	×
----	---

- a. Proposer une nouvelle implémentation possible (renommée **concatener_en_p(list1, list2)**) et expliquer en quoi elle paraît améliorer ou dégrader la précédente.
- b. On donne le code suivant :
- ```

L2 = Maillon(20, Maillon(30, None))
L1 = Maillon(10, L2)
L3 = Maillon(40, Maillon(50, None))
L4 = concatener_en_p(L1, L3)

```

Donner le contenu des listes L1, L2, L3 et L4 après l'exécution de chacun des instructions :

|                                     | L1 | L2 | L3 | L4 |
|-------------------------------------|----|----|----|----|
| L2 = Maillon(20, Maillon(30, None)) |    |    |    |    |
| L1 = Maillon(10, L2)                |    |    |    |    |
| L3 = Maillon(40, Maillon(50, None)) |    |    |    |    |
| L4 = concatener_en_p(L1, L3)        |    |    |    |    |

En quoi le code proposé est-il finalement risqué ?

- c. Quel serait le résultat de deux appels consécutifs à **L4 = concatener\_en\_p(L1, L3)** ?

#### IV. EXOS SUR LES LISTES

**Exo 10.** On donne le code de la fonction suivante :

```
def mystere(lst,v):
 """ Que fait-elle ????? """
 m=Maillon(v,None)
 if lst==None : return m
 maillon = lst
 while maillon.suivant is not None:
 maillon = maillon.suivant
 maillon.suivant=m
 return lst
```

Remplir le *docstring* de cette fonction et lui donner un nom moins mystérieux.

**Exo 11.** Coder la fonction suivante :

```
def renverser(lst):
 """renvoie une nouvelle liste inversion de la liste lst
 renverser (None) renvoie None
 renverser [30] renvoie [30]
 renverser [3, 6, 9, 100] renvoie [100, 9, 6, 3]
 """
```

**Exo 12.**

- Ecrire une fonction **trouve\_iter(x,lst)** qui renvoie le rang de la première occurrence de **x** dans **lst** si **x** s'y trouve, et **None** sinon.
- Ecrire la version récursive fonction **trouve\_rec(x,lst)**

**Exo 13.**

- Ecrire une fonction **occurences\_iter(x,lst)** qui renvoie le nombre d'occurrence de **x** dans **lst** si **x** s'y trouve, et **None** sinon.
- Ecrire la version récursive fonction **occurences\_rec(x,lst)**

**Exo 14.** Implémenter la fonction suivante :

```
def identiques(lst1, lst2):
 """retourne un booléen précisant si les listes lst1 et lst2 contiennent les
 mêmes éléments dans le même ordre"""
```

**Exo 15.**

- Ecrire une fonction récursive **insérer\_rec(x,lst)** qui prend en argument un entier **x** et une liste d'entier **lst**, supposée triée dans l'ordre croissant et qui renvoie une nouvelle liste dans laquelle **x** a été inséré à sa place. si **lst** est la liste **lst** contient dans cet ordre **1,2,5,8**, **insérer\_rec(3,lst)** contient dans cet ordre **1,2,3,5,8**
- Ecrire une fonction **tri\_par\_insertion(lst)** qui prend en argument une liste d'entier **lst**, et qui renvoie une nouvelle liste contenant les mêmes éléments triés dans l'ordre croissant. On suggère d'écrire une fonction récursive.

#### V. ENCAPSULATION DANS UN OBJET



On encapsule les listes chaînées dans des objets afin de masquer l'implémentation à l'utilisateur :

```
from classe_maillon import *
```

```
class Liste:
```

```
 """une liste chaînée"""
```

```
 def __init__(self):
```

```
 self.tete = None
```



la représentation d'une liste vide par None est cachée à l'utilisateur : on attend qu'il utilise la méthode `est_vide()` :

```
def est_vide(self):
 return self.tete is None
```



la construction de la liste se fait élément par élément, en accrochant des maillons successifs

```
def ajoute(self, x):
 self.tete = Maillon(x, self.tete)
```



accès au premier élément de la liste

```
def tete(self):
 return self.tete.valeur()
```

```
def queue(self):
 return self.tete.suite()
```

```
#permet l'appel str(L1) equivalent à L1.__str__()
def __str__(self, chaine=""):
 #fait appel à la méthode __str__() de la classe Maillon
 return str(self.tete)
```



la construction de la liste se fait élément par élément :

```
L1=Liste()
L1.ajoute(10) # [10]
L1.ajoute(20) # [20, 10]
L1.ajoute(30)
print(str(L1)) # [30, 20, 10]

#permet l'appel L1[i] equivalent à L1.__getitem__(i)
def __getitem__(self, n):
 #fait appel à la méthode nieme_element_rec() de la classe Maillon
 return nieme_element_rec(n, self.tete)
```



Le code complet est dans le fichier [classe\\_liste.py](#)

L'encapsulation de l'objet liste permet à l'utilisateur de s'affranchir de la notion de maillon et de la notion de chainage.

## VI. RETOUR SUR LES DICTIONNAIRES

**Les dictionnaires sont des objets qui peuvent en contenir d'autres.**

Au lieu d'héberger des informations dans un ordre précis comme les listes, un dictionnaire est une structure qui associe chaque objet contenu à une clé (l'objet peut être de tout type et la clé est généralement une chaîne de caractères, mais pas uniquement).

### a) Accès aux valeurs du dictionnaire

L'ordre des objets est indifférent.

Au lieu d'accéder à un élément par le biais d'un indice comme dans les listes, on accède à un élément du dictionnaire par la clé.

**Rq8.** Un dictionnaire peut être construit en donnant explicitement toutes ses entrées comme :  
**dicoAbs={"A":-2,"B":3,"C":5}.**

On peut aussi construire le même dictionnaire à partir d'un dictionnaire vide noté {} :

```
dicoAbs={}
dicoAbs["B"]=3
dicoAbs["A"]=-2
dicoAbs["C"]=5
```

**Rq 9.** L'ordre de construction n'est absolument pas pris en compte par l'interpréteur Python, comme le montre la réaction de la console ci-dessous :

```
>>> print(dicoAbs)
{'A': -2, 'C': 5, 'B': 3}
```

**Rq 10.** Comme pour les listes, tableaux ou tuples, le nombre d'entrées dans un dictionnaire est retourné par la fonction `len()` :

```
>>> len(dicoAbs)
3
```

## b) Test d'appartenance



On peut tester si le dictionnaire admet un élément avec une certaine clé :

```
>>> "A" in dicoAbs
True
>>> "K" in dicoAbs
False
>>> dicoAbs["K"]=(dicoAbs["A"]+dicoAbs["B"])/2 # insertion d'une nouvelle
entrée de clé « K »
>>> "K" in dicoAbs
True
>>> dicoAbs
{'A': -2, 'B': 3, 'C': 5, 'K': 0.5}
```

En python, les ensembles et les dictionnaires sont réalisées par des **tables de hachage**. Ces tables de hachages sont totalement masquées à l'utilisateur et on peut donc totalement en ignorer leur fonctionnement.

Il est cependant intéressant d'en retenir l'efficacité :

En pratique, on peut considérer qu'ajouter ou chercher dans un ensemble ou dans un dictionnaire est instantané quel que soit le nombre d'éléments  $n$  dans la structure.

On parle alors de **coût** (ou de **complexité**) en **temps constant** ou en  $O(1)$ .



La notation  $O(n)$  ramène à l'idée de **proportionnalité par rapport à  $n$** , donc égale à  $k \times n$  ( $k$  étant une constante)

La notation  $O(1)$  ramène à l'idée de **proportionnalité par rapport à 1**, donc égale à  $k$ , constante. On est alors en temps constant.

## Exo 16. En comparaison,

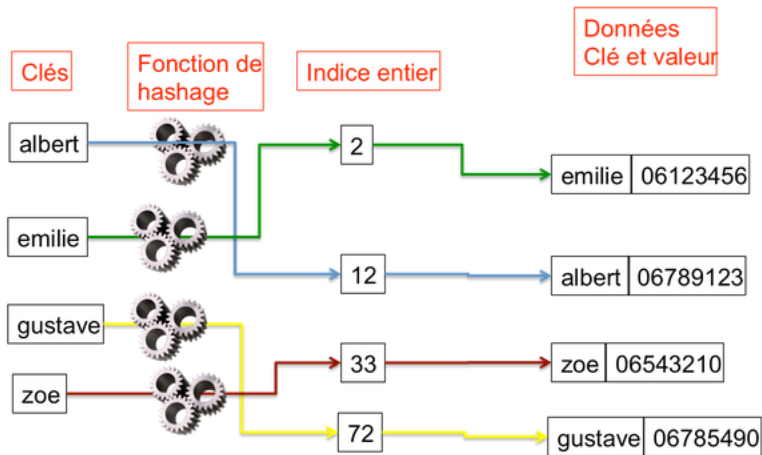
1. Dans un **tableau statique** de  $n$  éléments :
  - a. Quel est le coût pour ajouter un élément en tête d'un tableau statique ?  $O(n)$
  - b. Quel est le coût pour ajouter un élément en fin d'un tableau statique ?  $O(1)$
  - c. Quel est le coût pour chercher un élément dans d'un tableau statique ? en  $O(n)$  (pire des cas)
  - d.
  - e. Dans une **liste chaînée** de  $n$  éléments :
    - a. Quel est le coût pour ajouter un élément en tête d'une liste chaînée ? en  $O(1)$
    - b. Quel est le coût pour ajouter un élément en fin d'une liste chaînée ? en  $O(1)$
    - c. Quel est le coût pour chercher un élément dans une liste chaînée ?
    - d. Tables de hachage : <https://www.youtube.com/embed/lhJo8sXLfVw?feature=oembed>



## VII. COMPLÉMENT SUR LES FONCTIONS DE HASHAGE

Le premier principe essentiel des tables de hashage est de transformer une clé en un entier, qui sera alors l'indice du tableau contenant les données.

C'est le rôle de la fonction de hashage, qui assure une correspondance entre la clé et l'indice entier du tableau.



Ajouter un couple clé, valeur c'est

- Calculer l'indice que ce couple occupera dans la table, c'est à dire exécuter la fonction de hashage pour récupérer la valeur entière qu'elle retourne ;
- Mettre ce couple [clé, valeur] dans la table à l'indice qui vient d'être calculé

on calcule l'indice de hashage

$$\text{hash}(\text{"bac"}) = (2+1+3) \% 100 = 6$$

|     | Clé  | Valeur                       |
|-----|------|------------------------------|
| 0   |      |                              |
| ... |      |                              |
| 6   | bac  | examen passé en terminale    |
| ... |      |                              |
| 31  | lame | partie coupante d'un couteau |
| ... |      |                              |
| 42  |      |                              |
| ... |      |                              |
| 99  |      |                              |

La case d'indice 6 de la table n'est pas vide et contient la clé *bac* et sa définition

on calcule l'indice de hashage

$$\text{hash}(\text{"truc"}) = (20+18+1+3) \% 100 = 42$$

|     | Clé  | Valeur                       |
|-----|------|------------------------------|
| 0   |      |                              |
| ... |      |                              |
| 6   | bac  | examen passé en terminale    |
| ... |      |                              |
| 31  | lame | partie coupante d'un couteau |
| ... |      |                              |
| 42  |      |                              |
| ... |      |                              |
| 99  |      |                              |

La case d'indice 42 de la table est vide et la table ne contient aucune information sur le mot "truc".